

Course- BTech
Course Code- CSET101
Year- 2023
Date-

Type- Core
Course Name- Computational Thinking
and Programming
Semester- odd
Batch- ALL

Tutorial Assignment 12

Tutorial title: Graphics, File handling and Miscellaneous

CO Mapping

Name	CO1	CO2	CO3
Loops, Sequence Operations	✓	✓	✓

EXERCISE

1. Write a python code to draw an equilateral triangle using turtle.
2. Write a python file handling program to display from the 10th character to 30th character of a file.
3. Predict the output of the following code.

```
class India():
    def capital(self):
        print("New Delhi is the capital of India.")

    def language(self):
        print("Hindi is the most widely spoken language of India.")

    def type(self):
        print("India is a developing country.")

class USA():
    def capital(self):
        print("Washington, D.C. is the capital of USA.")

    def language(self):
        print("English is the primary language of USA.")

    def type(self):
        print("USA is a developed country.")

obj_ind = India()
obj_usa = USA()
for country in (obj_ind, obj_usa):
    country.capital()
    country.language ()
    country.type()
```

4. Predict the output of following code.

```
class Animal:
    def __init__(self, AnimalName):
        print(AnimalName, 'lives in a farm.')

class Cow(Animal):
    def __init__(self):
        print('Cow gives us milk.')
        super().__init__('Cow')

result = Cow()
res1=Animal("Horse")
```

5. Predict the output of following code:

```
class SpecialNumbers:
    euler_constant = 0.577
    euler_number = 2.718
    pi_number = 3.14
    golden_ratio = 1.618
    msg = 'These numbers are special.'
special_numbers = SpecialNumbers()
print('The euler number is', getattr(special_numbers, 'euler_number'))
print('The golden ratio is', special_numbers.golden_ratio)
setattr(special_numbers, 'pi_number', 3.142371)
print('The pi number is', getattr(special_numbers, 'pi_number'))
print(hasattr(SpecialNumbers, 'msg'))
delattr(SpecialNumbers, 'msg')
print('The message is ', getattr(special_numbers, 'msg'))
```

6. Predict the output of following question

```
class Student:
    """ This is student class"""
    def __init__(self, name, id, age):
        self.name = name;
        self.id = id;
        self.age = age

    def display_details(self):
        print("Name:%s, ID:%d, age:%d"%(self.name, self.id))

s = Student("Amit", 101, 22)
print(s.__doc__)
```

```
print(s.__dict__)  
print(s.__module__)
```

7. What will be the output of the following Python code?

```
import turtle  
t=turtle.Pen()  
for i in range(0,4):  
    t.forward(100)  
    t.left(90)  
  
t.penup()  
t.left(90)  
t.forward(200)  
for i in range(0,4):  
    t.forward(100)  
    t.left(90)
```

- a) Error
- b) 1 square
- c) 2 squares, at a separation of 100 units, joined by a straight line
- d) 2 squares, at a separation of 100 units, without a line joining them

8. What will be the output of the following Python code?

```
import turtle  
t=turtle.Pen()  
for i in range(0,5):  
    t.left(144)  
    t.forward(100)
```

- a) Trapezium
- b) Parallelepiped
- c) Tetrahedron
- d) Star

9. What will be the output of the following Python code?

```
import turtle  
t=turtle.Pen()  
t.backward(100)  
t.penup()  
t.right(45)  
print(t.isdown())
```

- a) No Output
- b) False
- c) Error
- d) No

10 A) In which direction is the turtle pointed by default?

- a) North
- b) South
- c) East
- d) West

B) Which of the following functions does not accept any arguments?

- a) position()
- b) fillcolor()
- c) goto()
- d) setheading()

11. What will be the output of the following Python functions?

```
import turtle
t=turtle.Pen()
for i in range(0,3):
    t.forward(100)
    t.left(120)

t.back(100)
for i in range(0,3):
    t.forward(100)
    t.left(120)
```

- a) Error
- b) Two triangles, joined by a straight line
- c) Two triangles, joined at one vertex
- d) Two separate triangles, not connected by a line



tkinter — Python interface to Tcl/Tk

Source code: [Lib/tkinter/__init__.py](#)

The `tkinter` package (“Tk interface”) is the standard Python interface to the Tcl/Tk GUI toolkit. Both Tk and `tkinter` are available on most Unix platforms, including macOS, as well as on Windows systems.

Running `python -m tkinter` from the command line should open a window demonstrating a simple Tk interface, letting you know that `tkinter` is properly installed on your system, and also showing what version of Tcl/Tk is installed, so you can read the Tcl/Tk documentation specific to that version.

Tkinter supports a range of Tcl/Tk versions, built either with or without thread support. The official Python binary release bundles Tcl/Tk 8.6 threaded. See the source code for the `_tkinter` module for more information about supported versions.

Tkinter is not a thin wrapper, but adds a fair amount of its own logic to make the experience more pythonic. This documentation will concentrate on these additions and changes, and refer to the official Tcl/Tk documentation for details that are unchanged.

Note: Tcl/Tk 8.5 (2007) introduced a modern set of themed user interface components along with a new API to use them. Both old and new APIs are still available. Most documentation you will find online still uses the old API and can be woefully outdated.

See also:

- [TkDocs](#)

Extensive tutorial on creating user interfaces with Tkinter. Explains key concepts, and illustrates recommended approaches using the modern API.

- [Tkinter 8.5 reference: a GUI for Python](#)

Reference documentation for Tkinter 8.5 detailing available classes, methods, and options.

Tcl/Tk Resources:

- [Tk commands](#)

Comprehensive reference to each of the underlying Tcl/Tk commands used by Tkinter.

- [Tcl/Tk Home Page](#)

Additional documentation, and links to Tcl/Tk core development.

Books:

- [Modern Tkinter for Busy Python Developers](#)

By Mark Roseman. (ISBN 978-1999149567)

- [Python and Tkinter Programming](#)

By Alan Moore. (ISBN 978-1788835886)

- [Programming Python](#)

By Mark Lutz; has excellent coverage of Tkinter. (ISBN 978-0596158101)



By John Ousterhout, inventor of Tcl/Tk, and Ken Jones; does not cover Tkinter. (ISBN 978-0321336330)

Architecture

Tcl/Tk is not a single library but rather consists of a few distinct modules, each with separate functionality and its own official documentation. Python's binary releases also ship an add-on module together with it.

Tcl

Tcl is a dynamic interpreted programming language, just like Python. Though it can be used on its own as a general-purpose programming language, it is most commonly embedded into C applications as a scripting engine or an interface to the Tk toolkit. The Tcl library has a C interface to create and manage one or more instances of a Tcl interpreter, run Tcl commands and scripts in those instances, and add custom commands implemented in either Tcl or C. Each interpreter has an event queue, and there are facilities to send events to it and process them. Unlike Python, Tcl's execution model is designed around cooperative multitasking, and Tkinter bridges this difference (see [Threading model](#) for details).

Tk

Tk is a [Tcl package](#) implemented in C that adds custom commands to create and manipulate GUI widgets. Each [Tk](#) object embeds its own Tcl interpreter instance with Tk loaded into it. Tk's widgets are very customizable, though at the cost of a dated appearance. Tk uses Tcl's event queue to generate and process GUI events.

Ttk

Themed Tk (Ttk) is a newer family of Tk widgets that provide a much better appearance on different platforms than many of the classic Tk widgets. Ttk is distributed as part of Tk, starting with Tk version 8.5. Python bindings are provided in a separate module, [tkinter.ttk](#).

Internally, Tk and Ttk use facilities of the underlying operating system, i.e., Xlib on Unix/X11, Cocoa on macOS, GDI on Windows.

When your Python application uses a class in Tkinter, e.g., to create a widget, the [tkinter](#) module first assembles a Tcl/Tk command string. It passes that Tcl command string to an internal `_tkinter` binary module, which then calls the Tcl interpreter to evaluate it. The Tcl interpreter will then call into the Tk and/or Ttk packages, which will in turn make calls to Xlib, Cocoa, or GDI.

Tkinter Modules

Support for Tkinter is spread across several modules. Most applications will need the main [tkinter](#) module, as well as the [tkinter.ttk](#) module, which provides the modern themed widget set and API:

```
from tkinter import *
from tkinter import ttk
```

```
class tkinter.Tk(screenName=None, baseName=None, className='Tk', useTk=True, sync=False, use=None)
```

Construct a toplevel Tk widget, which is usually the main window of an application, and initialize a Tcl interpreter for this widget. Each instance has its own associated Tcl interpreter.

The [Tk](#) class is typically instantiated using all default values. However, the following keyword arguments are currently recognized:

screenName

**baseName**

Name of the profile file. By default, *baseName* is derived from the program name (`sys.argv[0]`).

className

Name of the widget class. Used as a profile file and also as the name with which Tcl is invoked (*argv0* in *interp*).

useTk

If True, initialize the Tk subsystem. The `tkinter.Tcl()` function sets this to False.

sync

If True, execute all X server commands synchronously, so that errors are reported immediately. Can be used for debugging. (X11 only)

use

Specifies the *id* of the window in which to embed the application, instead of it being created as an independent toplevel window. *id* must be specified in the same way as the value for the `-use` option for toplevel widgets (that is, it has a form like that returned by `wininfo_id()`).

Note that on some platforms this will only work correctly if *id* refers to a Tk frame or toplevel that has its `-container` option enabled.

Tk reads and interprets profile files, named `.className.tcl` and `.baseName.tcl`, into the Tcl interpreter and calls `exec()` on the contents of `.className.py` and `.baseName.py`. The path for the profile files is the HOME environment variable or, if that isn't defined, then `os.curdir`.

tk

The Tk application object created by instantiating `Tk`. This provides access to the Tcl interpreter. Each widget that is attached the same instance of `Tk` has the same value for its `tk` attribute.

master

The widget object that contains this widget. For `Tk`, the *master* is `None` because it is the main window. The terms *master* and *parent* are similar and sometimes used interchangeably as argument names; however, calling `wininfo_parent()` returns a string of the widget name whereas `master` returns the object. *parent/child* reflects the tree-like relationship while *master/slave* reflects the container structure.

children

The immediate descendants of this widget as a `dict` with the child widget names as the keys and the child instance objects as the values.

`tkinter.Tcl(screenName=None, baseName=None, className='Tk', useTk=False)`

The `Tcl()` function is a factory function which creates an object much like that created by the `Tk` class, except that it does not initialize the Tk subsystem. This is most often useful when driving the Tcl interpreter in an environment where one doesn't want to create extraneous toplevel windows, or where one cannot (such as Unix/Linux systems without an X server). An object created by the `Tcl()` object can have a Toplevel window created (and the Tk subsystem initialized) by calling its `loadtk()` method.

The modules that provide Tk support include:

tkinter

Main Tkinter module.



`tkinter.commondialog`

Base class for the dialogs defined in the other modules listed here.

`tkinter.filedialog`

Common dialogs to allow the user to specify a file to open or save.

`tkinter.font`

Utilities to help work with fonts.

`tkinter.messagebox`

Access to standard Tk dialog boxes.

`tkinter.scrolledtext`

Text widget with a vertical scroll bar built in.

`tkinter.simpdialog`

Basic dialogs and convenience functions.

`tkinter.ttk`

Themed widget set introduced in Tk 8.5, providing modern alternatives for many of the classic widgets in the main `tkinter` module.

Additional modules:

`_tkinter`

A binary module that contains the low-level interface to Tcl/Tk. It is automatically imported by the main `tkinter` module, and should never be used directly by application programmers. It is usually a shared library (or DLL), but might in some cases be statically linked with the Python interpreter.

`idlelib`

Python's Integrated Development and Learning Environment (IDLE). Based on `tkinter`.

`tkinter.constants`

Symbolic constants that can be used in place of strings when passing various parameters to Tkinter calls. Automatically imported by the main `tkinter` module.

`tkinter.dnd`

(experimental) Drag-and-drop support for `tkinter`. This will become deprecated when it is replaced with the Tk DND.

`tkinter.tix`

(deprecated) An older third-party Tcl/Tk package that adds several new widgets. Better alternatives for most can be found in `tkinter.ttk`.

`turtle`

Turtle graphics in a Tk window.

Tkinter Life Preserver

This section is not designed to be an exhaustive tutorial on either Tk or Tkinter. For that, refer to one of the external resources noted earlier. Instead, this section provides a very quick orientation to what a Tkinter application looks like, identifies foundational Tk concepts, and explains how the Tkinter wrapper is structured.

The remainder of this section will help you to identify the classes, methods, and options you'll need in your Tkinter application, and where to find more detailed documentation on them, including in the official Tcl/Tk



A Hello World Program

We'll start by walking through a "Hello World" application in Tkinter. This isn't the smallest one we could write, but has enough to illustrate some key concepts you'll need to know.

```
from tkinter import *
from tkinter import ttk
root = Tk()
frm = ttk.Frame(root, padding=10)
frm.grid()
ttk.Label(frm, text="Hello World!").grid(column=0, row=0)
ttk.Button(frm, text="Quit", command=root.destroy).grid(column=1, row=0)
root.mainloop()
```

After the imports, the next line creates an instance of the `Tk` class, which initializes Tk and creates its associated Tcl interpreter. It also creates a toplevel window, known as the root window, which serves as the main window of the application.

The following line creates a frame widget, which in this case will contain a label and a button we'll create next. The frame is fit inside the root window.

The next line creates a label widget holding a static text string. The `grid()` method is used to specify the relative layout (position) of the label within its containing frame widget, similar to how tables in HTML work.

A button widget is then created, and placed to the right of the label. When pressed, it will call the `destroy()` method of the root window.

Finally, the `mainloop()` method puts everything on the display, and responds to user input until the program terminates.

Important Tk Concepts

Even this simple program illustrates the following key Tk concepts:

widgets

A Tkinter user interface is made up of individual *widgets*. Each widget is represented as a Python object, instantiated from classes like `ttk.Frame`, `ttk.Label`, and `ttk.Button`.

widget hierarchy

Widgets are arranged in a *hierarchy*. The label and button were contained within a frame, which in turn was contained within the root window. When creating each *child* widget, its *parent* widget is passed as the first argument to the widget constructor.

configuration options

Widgets have *configuration options*, which modify their appearance and behavior, such as the text to display in a label or button. Different classes of widgets will have different sets of options.

geometry management

Widgets aren't automatically added to the user interface when they are created. A *geometry manager* like `grid` controls where in the user interface they are placed.

event loop

Tkinter reacts to user input, changes from your program, and even refreshes the display only when actively running an *event loop*. If your program isn't running the event loop, your user interface won't update.



When your application uses Tkinter's classes and methods, internally Tkinter is assembling strings representing Tcl/Tk commands, and executing those commands in the Tcl interpreter attached to your application's `Tk` instance.

Whether it's trying to navigate reference documentation, trying to find the right method or option, adapting some existing code, or debugging your Tkinter application, there are times that it will be useful to understand what those underlying Tcl/Tk commands look like.

To illustrate, here is the Tcl/Tk equivalent of the main part of the Tkinter script above.

```
ttk::frame .frm -padding 10
grid .frm
grid [ttk::label .frm.lbl -text "Hello World!"] -column 0 -row 0
grid [ttk::button .frm.btn -text "Quit" -command "destroy ."] -column 1 -row 0
```

Tcl's syntax is similar to many shell languages, where the first word is the command to be executed, with arguments to that command following it, separated by spaces. Without getting into too many details, notice the following:

- The commands used to create widgets (like `ttk::frame`) correspond to widget classes in Tkinter.
- Tcl widget options (like `-text`) correspond to keyword arguments in Tkinter.
- Widgets are referred to by a *pathname* in Tcl (like `.frm.btn`), whereas Tkinter doesn't use names but object references.
- A widget's place in the widget hierarchy is encoded in its (hierarchical) pathname, which uses a `.` (dot) as a path separator. The pathname for the root window is just `.` (dot). In Tkinter, the hierarchy is defined not by pathname but by specifying the parent widget when creating each child widget.
- Operations which are implemented as separate *commands* in Tcl (like `grid` or `destroy`) are represented as *methods* on Tkinter widget objects. As you'll see shortly, at other times Tcl uses what appear to be method calls on widget objects, which more closely mirror what would be used in Tkinter.

How do I...? What option does...?

If you're not sure how to do something in Tkinter, and you can't immediately find it in the tutorial or reference documentation you're using, there are a few strategies that can be helpful.

First, remember that the details of how individual widgets work may vary across different versions of both Tkinter and Tcl/Tk. If you're searching documentation, make sure it corresponds to the Python and Tcl/Tk versions installed on your system.

When searching for how to use an API, it helps to know the exact name of the class, option, or method that you're using. Introspection, either in an interactive Python shell or with `print()`, can help you identify what you need.

To find out what configuration options are available on any widget, call its `configure()` method, which returns a dictionary containing a variety of information about each object, including its default and current values. Use `keys()` to get just the names of each option.

```
btn = ttk.Button(frm, ...)
print(btn.configure().keys())
```

As most widgets have many configuration options in common, it can be useful to find out which are specific to a particular widget class. Comparing the list of options to that of a simpler widget, like a frame, is one way to do that.



Similarly, you can find the available methods for a widget object using the standard `dir()` function. If you try it, you'll see there are over 200 common widget methods, so again identifying those specific to a widget class is helpful.

```
print(dir(btn))
print(set(dir(btn)) - set(dir(frm)))
```

Navigating the Tcl/Tk Reference Manual

As noted, the official [Tk commands](#) reference manual (man pages) is often the most accurate description of what specific operations on widgets do. Even when you know the name of the option or method that you need, you may still have a few places to look.

While all operations in Tkinter are implemented as method calls on widget objects, you've seen that many Tcl/Tk operations appear as commands that take a widget pathname as its first parameter, followed by optional parameters, e.g.

```
destroy .
grid .frm.btn -column 0 -row 0
```

Others, however, look more like methods called on a widget object (in fact, when you create a widget in Tcl/Tk, it creates a Tcl command with the name of the widget pathname, with the first parameter to that command being the name of a method to call).

```
.frm.btn invoke
.frm.lbl configure -text "Goodbye"
```

In the official Tcl/Tk reference documentation, you'll find most operations that look like method calls on the man page for a specific widget (e.g., you'll find the `invoke()` method on the [ttk::button](#) man page), while functions that take a widget as a parameter often have their own man page (e.g., [grid](#)).

You'll find many common options and methods in the [options](#) or [ttk::widget](#) man pages, while others are found in the man page for a specific widget class.

You'll also find that many Tkinter methods have compound names, e.g., `wininfo_x()`, `wininfo_height()`, `wininfo_viewable()`. You'd find documentation for all of these in the [wininfo](#) man page.

Note: Somewhat confusingly, there are also methods on all Tkinter widgets that don't actually operate on the widget, but operate at a global scope, independent of any widget. Examples are methods for accessing the clipboard or the system bell. (They happen to be implemented as methods in the base `Widget` class that all Tkinter widgets inherit from).

Threading model

Python and Tcl/Tk have very different threading models, which [tkinter](#) tries to bridge. If you use threads, you may need to be aware of this.

A Python interpreter may have many threads associated with it. In Tcl, multiple threads can be created, but each thread has a separate Tcl interpreter instance associated with it. Threads can also create more than one interpreter instance, though each interpreter instance can be used only by the one thread that created it.

Each [Tk](#) object created by [tkinter](#) contains a Tcl interpreter. It also keeps track of which thread created that interpreter. Calls to [tkinter](#) can be made from any Python thread. Internally, if a call comes from a thread



Tcl/Tk applications are normally event-driven, meaning that after initialization, the interpreter runs an event loop (i.e. `Tk.mainloop()`) and responds to events. Because it is single-threaded, event handlers must respond quickly, otherwise they will block other events from being processed. To avoid this, any long-running computations should not run in an event handler, but are either broken into smaller pieces using timers, or run in another thread. This is different from many GUI toolkits where the GUI runs in a completely separate thread from all application code including event handlers.

If the Tcl interpreter is not running the event loop and processing events, any `tkinter` calls made from threads other than the one running the Tcl interpreter will fail.

A number of special cases exist:

- Tcl/Tk libraries can be built so they are not thread-aware. In this case, `tkinter` calls the library from the originating Python thread, even if this is different than the thread that created the Tcl interpreter. A global lock ensures only one call occurs at a time.
- While `tkinter` allows you to create more than one instance of a `Tk` object (with its own interpreter), all interpreters that are part of the same thread share a common event queue, which gets ugly fast. In practice, don't create more than one instance of `Tk` at a time. Otherwise, it's best to create them in separate threads and ensure you're running a thread-aware Tcl/Tk build.
- Blocking event handlers are not the only way to prevent the Tcl interpreter from reentering the event loop. It is even possible to run multiple nested event loops or abandon the event loop entirely. If you're doing anything tricky when it comes to events or threads, be aware of these possibilities.
- There are a few select `tkinter` functions that presently work only when called from the thread that created the Tcl interpreter.

Handy Reference

Setting Options

Options control things like the color and border width of a widget. Options can be set in three ways:

At object creation time, using keyword arguments

```
fred = Button(self, fg="red", bg="blue")
```

After object creation, treating the option name like a dictionary index

```
fred["fg"] = "red"  
fred["bg"] = "blue"
```

Use the `config()` method to update multiple attrs subsequent to object creation

```
fred.config(fg="red", bg="blue")
```

For a complete explanation of a given option and its behavior, see the Tk man pages for the widget in question.

Note that the man pages list “STANDARD OPTIONS” and “WIDGET SPECIFIC OPTIONS” for each widget. The former is a list of options that are common to many widgets, the latter are the options that are idiosyncratic to that particular widget. The Standard Options are documented on the [options\(3\)](#) man page.

No distinction between standard and widget-specific options is made in this document. Some options don't apply to some kinds of widgets. Whether a given widget responds to a particular option depends on the class of the widget; buttons have a `command` option, labels do not.



value of these calls is a dictionary whose key is the name of the option as a string (for example, 'relief') and whose values are 5-tuples.

Some options, like `bg` are synonyms for common options with long names (`bg` is shorthand for “background”). Passing the `config()` method the name of a shorthand option will return a 2-tuple, not 5-tuple. The 2-tuple passed back will contain the name of the synonym and the “real” option (such as ('bg', 'background')).

Index	Meaning	Example
0	option name	'relief'
1	option name for database lookup	'relief'
2	option class for database lookup	'Relief'
3	default value	'raised'
4	current value	'groove'

Example:

```
>>> print(fred.config())
{'relief': ('relief', 'relief', 'Relief', 'raised', 'groove')}
```

```
>>>
```

Of course, the dictionary printed will include all the options available and their values. This is meant only as an example.

The Packer

The packer is one of Tk’s geometry-management mechanisms. Geometry managers are used to specify the relative positioning of widgets within their container - their mutual *master*. In contrast to the more cumbersome *placer* (which is used less commonly, and we do not cover here), the packer takes qualitative relationship specification - *above*, *to the left of*, *filling*, etc - and works everything out to determine the exact placement coordinates for you.

The size of any *master* widget is determined by the size of the “slave widgets” inside. The packer is used to control where slave widgets appear inside the master into which they are packed. You can pack widgets into frames, and frames into other frames, in order to achieve the kind of layout you desire. Additionally, the arrangement is dynamically adjusted to accommodate incremental changes to the configuration, once it is packed.

Note that widgets do not appear until they have had their geometry specified with a geometry manager. It’s a common early mistake to leave out the geometry specification, and then be surprised when the widget is created but nothing appears. A widget will appear only after it has had, for example, the packer’s `pack()` method applied to it.

The `pack()` method can be called with keyword-option/value pairs that control where the widget is to appear within its container, and how it is to behave when the main application window is resized. Here are some examples:

```
fred.pack()                # defaults to side = "top"
fred.pack(side="left")
fred.pack(expand=1)
```

Packer Options



anchor

Anchor type. Denotes where the packer is to place each slave in its parcel.

expand

Boolean, 0 or 1.

fill

Legal values: 'x', 'y', 'both', 'none'.

ipadx and ipady

A distance - designating internal padding on each side of the slave widget.

padx and pady

A distance - designating external padding on each side of the slave widget.

side

Legal values are: 'left', 'right', 'top', 'bottom'.

Coupling Widget Variables

The current-value setting of some widgets (like text entry widgets) can be connected directly to application variables by using special options. These options are `variable`, `textvariable`, `onvalue`, `offvalue`, and `value`. This connection works both ways: if the variable changes for any reason, the widget it's connected to will be updated to reflect the new value.

Unfortunately, in the current implementation of `tkinter` it is not possible to hand over an arbitrary Python variable to a widget through a `variable` or `textvariable` option. The only kinds of variables for which this works are variables that are subclassed from a class called `Variable`, defined in `tkinter`.

There are many useful subclasses of `Variable` already defined: `StringVar`, `IntVar`, `DoubleVar`, and `BooleanVar`. To read the current value of such a variable, call the `get()` method on it, and to change its value you call the `set()` method. If you follow this protocol, the widget will always track the value of the variable, with no further intervention on your part.

For example:

```
import tkinter as tk

class App(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.pack()

        self.entrythingy = tk.Entry()
        self.entrythingy.pack()

        # Create the application variable.
        self.contents = tk.StringVar()
        # Set it to some value.
        self.contents.set("this is a variable")
        # Tell the entry widget to watch this variable.
        self.entrythingy["textvariable"] = self.contents

        # Define a callback for when the user hits return.
        # It prints the current value of the variable.
        self.entrythingy.bind('<Key-Return>',
                              self.print_contents)
```



```

print(f"the current entry content is: ",
      self.contents.get())

root = tk.Tk()
myapp = App(root)
myapp.mainloop()

```

The Window Manager

In Tk, there is a utility command, `wm`, for interacting with the window manager. Options to the `wm` command allow you to control things like titles, placement, icon bitmaps, and the like. In `tkinter`, these commands have been implemented as methods on the `Wm` class. Toplevel widgets are subclassed from the `Wm` class, and so can call the `Wm` methods directly.

To get at the toplevel window that contains a given widget, you can often just refer to the widget's master. Of course if the widget has been packed inside of a frame, the master won't represent a toplevel window. To get at the toplevel window that contains an arbitrary widget, you can call the `_root()` method. This method begins with an underscore to denote the fact that this function is part of the implementation, and not an interface to Tk functionality.

Here are some examples of typical usage:

```

import tkinter as tk

class App(tk.Frame):
    def __init__(self, master=None):
        super().__init__(master)
        self.pack()

# create the application
myapp = App()

#
# here are method calls to the window manager class
#
myapp.master.title("My Do-Nothing Application")
myapp.master.maxsize(1000, 400)

# start the program
myapp.mainloop()

```

Tk Option Data Types

anchor

Legal values are points of the compass: "n", "ne", "e", "se", "s", "sw", "w", "nw", and also "center".

bitmap

There are eight built-in, named bitmaps: 'error', 'gray25', 'gray50', 'hourglass', 'info', 'questhead', 'question', 'warning'. To specify an X bitmap filename, give the full path to the file, preceded with an @, as in "@usr/contrib/bitmap/gumby.bit".

boolean

You can pass integers 0 or 1 or the strings "yes" or "no".

callback

This is any Python function that takes no arguments. For example:



```
fred["command"] = print_it
```

color

Colors can be given as the names of X colors in the `rgb.txt` file, or as strings representing RGB values in 4 bit: `"#RGB"`, 8 bit: `"#RRGGBB"`, 12 bit: `"#RRRGGBBBB"`, or 16 bit: `"#RRRRGGGGBBBB"` ranges, where R,G,B here represent any legal hex digit. See page 160 of Ousterhout's book for details.

cursor

The standard X cursor names from `cursorfont.h` can be used, without the `XC_` prefix. For example to get a hand cursor (`XC_hand2`), use the string `"hand2"`. You can also specify a bitmap and mask file of your own. See page 179 of Ousterhout's book.

distance

Screen distances can be specified in either pixels or absolute distances. Pixels are given as numbers and absolute distances as strings, with the trailing character denoting units: `c` for centimetres, `i` for inches, `m` for millimetres, `p` for printer's points. For example, 3.5 inches is expressed as `"3.5i"`.

font

Tk uses a list font name format, such as `{courier 10 bold}`. Font sizes with positive numbers are measured in points; sizes with negative numbers are measured in pixels.

geometry

This is a string of the form `widthxheight`, where width and height are measured in pixels for most widgets (in characters for widgets displaying text). For example: `fred["geometry"] = "200x100"`.

justify

Legal values are the strings: `"left"`, `"center"`, `"right"`, and `"fill"`.

region

This is a string with four space-delimited elements, each of which is a legal distance (see above). For example: `"2 3 4 5"` and `"3i 2i 4.5i 2i"` and `"3c 2c 4c 10.43c"` are all legal regions.

relief

Determines what the border style of a widget will be. Legal values are: `"raised"`, `"sunken"`, `"flat"`, `"groove"`, and `"ridge"`.

scrollcommand

This is almost always the `set()` method of some scrollbar widget, but can be any widget method that takes a single argument.

wrap

Must be one of: `"none"`, `"char"`, or `"word"`.

Bindings and Events

The `bind` method from the widget command allows you to watch for certain events and to have a callback function trigger when that event type occurs. The form of the `bind` method is:

```
def bind(self, sequence, func, add='')
```

where:

sequence

is a string that denotes the target kind of event. (See the [bind\(3tk\)](#) man page, and page 201 of John Ousterhout's book, *Tcl and the Tk Toolkit (2nd edition)*, for details).



be passed as the argument. (Functions deployed this way are commonly known as *callbacks*.)

`add`

is optional, either `''` or `'+'`. Passing an empty string denotes that this binding is to replace any other bindings that this event is associated with. Passing a `'+'` means that this function is to be added to the list of functions bound to this event type.

For example:

```
def turn_red(self, event):
    event.widget["activeforeground"] = "red"

self.button.bind("<Enter>", self.turn_red)
```

Notice how the `widget` field of the event is being accessed in the `turn_red()` callback. This field contains the widget that caught the X event. The following table lists the other event fields you can access, and how they are denoted in Tk, which can be useful when referring to the Tk man pages.

Tk	Tkinter Event Field	Tk	Tkinter Event Field
%f	focus	%A	char
%h	height	%E	send_event
%k	keycode	%K	keysym
%s	state	%N	keysym_num
%t	time	%T	type
%w	width	%W	widget
%x	x	%X	x_root
%y	y	%Y	y_root

The index Parameter

A number of widgets require “index” parameters to be passed. These are used to point at a specific place in a Text widget, or to particular characters in an Entry widget, or to particular menu items in a Menu widget.

Entry widget indexes (index, view index, etc.)

Entry widgets have options that refer to character positions in the text being displayed. You can use these [tkinter](#) functions to access these special points in text widgets:

Text widget indexes

The index notation for Text widgets is very rich and is best described in the Tk man pages.

Menu indexes (menu.invoke(), menu.entryconfig(), etc.)

Some options and methods for menus manipulate specific menu entries. Anytime a menu index is needed for an option or a parameter, you may pass in:

- an integer which refers to the numeric position of the entry in the widget, counted from the top, starting with 0;
- the string "active", which refers to the menu position that is currently under the cursor;
- the string "last" which refers to the last menu item;



- the string "none", which indicates no menu entry at all, most often used with `menu.activate()` to deactivate all entries, and finally,
- a text string that is pattern matched against the label of the menu entry, as scanned from the top of the menu to the bottom. Note that this index type is considered after all the others, which means that matches for menu items labelled `last`, `active`, or `none` may be interpreted as the above literals, instead.

Images

Images of different formats can be created through the corresponding subclass of `tkinter.Image`:

- `BitmapImage` for images in XBM format.
- `PhotoImage` for images in PGM, PPM, GIF and PNG formats. The latter is supported starting with Tk 8.6.

Either type of image is created through either the `file` or the `data` option (other options are available as well).

The image object can then be used wherever an `image` option is supported by some widget (e.g. labels, buttons, menus). In these cases, Tk will not keep a reference to the image. When the last Python reference to the image object is deleted, the image data is deleted as well, and Tk will display an empty box wherever the image was used.

See also: The [Pillow](#) package adds support for formats such as BMP, JPEG, TIFF, and WebP, among others.

File Handlers

Tk allows you to register and unregister a callback function which will be called from the Tk mainloop when I/O is possible on a file descriptor. Only one handler may be registered per file descriptor. Example code:

```
import tkinter
widget = tkinter.Tk()
mask = tkinter.READABLE | tkinter.WRITABLE
widget.tk.createfilehandler(file, mask, callback)
...
widget.tk.deletefilehandler(file)
```

This feature is not available on Windows.

Since you don't know how many bytes are available for reading, you may not want to use the [BufferedIOBase](#) or [TextIOBase](#) `read()` or `readline()` methods, since these will insist on reading a predefined number of bytes. For sockets, the `recv()` or `recvfrom()` methods will work fine; for other files, use raw reads or `os.read(file.fileno(), maxbytecount)`.

`Widget.tk.createfilehandler(file, mask, func)`

Registers the file handler callback function *func*. The *file* argument may either be an object with a `fileno()` method (such as a file or socket object), or an integer file descriptor. The *mask* argument is an ORed combination of any of the three constants below. The callback is called as follows:

```
callback(file, mask)
```

`Widget.tk.deletefilehandler(file)`

Unregisters a file handler.

`tkinter.READABLE`





Constants used in the *mask* arguments.